

PROJECT DEVELOPMENT PHASE

ComicCraft – AI Comic Story Creator using Gemini Models

1. Introduction

The Project Development Phase is the stage where the planned ComicCraft design is converted into working software. ComicCraft is a web-based application that generates personalized comic stories, dialogues, and illustrations from user-provided prompts. The development combines a FastAPI backend, HTML/CSS/Jinja2 frontend, Google Gemini models, Stable Diffusion, and PDF generation.

2. Development Objectives

- Develop the core AI-powered comic generation functions.
- Implement the FastAPI backend for routing and user-input processing.
- Connect Gemini Flash and Gemini Pro to the application workflow.
- Integrate Stable Diffusion for comic-style image generation.
- Build and organize comic panels with images and story content.
- Export the completed comic as a multi-page PDF.
- Connect backend processing with the frontend templates.

3. Development Workflow

The development follows a connected workflow. User information is collected from the web interface. The FastAPI backend receives the information and invokes the appropriate AI functions. Gemini Flash creates a structured comic outline, Gemini Pro expands it into narration and dialogue, and Stable Diffusion generates illustrations. The generated content is then organized into a layout and exported as a PDF.

4. Core Functionality Development

Activity 2.1 – Develop the Core Functionalities

The core development work implements the main functions required to transform a user's creative idea into a complete comic. Each function has a specific responsibility in the generation pipeline.

1. Generate Structured Comic Panel Outline

Function: `generate_outline()` **File:** `gemini_flash.py`

This function uses the Gemini Flash AI model to generate a structured five-panel comic outline from the user's story prompt. Each panel contains a panel number, title, scene description, and image generation prompt. The output is organized as a list of dictionaries, with one dictionary for each panel.

2. Generate Comic Story Narration and Dialogue

Function: `generate_story()` **File:** `gemini_pro.py`

This function uses Gemini Pro to expand the panel outlines into a complete comic-style story. It creates narration and character dialogues for the panels and returns the story as formatted text.

3. Generate Comic Illustrations

Function: `generate_image()` **File:** `image_generator.py`

This function uses Stable Diffusion to create comic-style images from the image prompts. The generated image is saved in the `static/panels` directory, and the function returns its file path.

5. Layout, PDF Export and Backend Development

4. Organize Comic Panels

Function: `build_comic_layout()` **File:** `layout_builder.py`

This function organizes the generated images and the complete comic story into a structured layout. It matches each image with its corresponding panel story. The resulting layout contains the panel number, image path, and text.

5. Export Comic into PDF

Function: `save_pdf()` **File:** `exporters.py`

The `save_pdf()` function compiles the complete comic into a multi-page PDF using the FPDF library. Each panel's image and narration are placed on separate pages. The final PDF is saved in the `static/exports` folder.

6. FastAPI Backend Implementation

The FastAPI backend manages routing, receives user inputs, calls the AI generation functions, organizes the final comic, and supports PDF export. Form data is collected through `Form(...)` parameters, while JSON-based comic generation uses a Pydantic `PromptRequest` schema.

7. Main Generation Process

- Receive story prompt, character name, setting, tone, and art style.
- Call `generate_outline()` to create the five-panel structure.
- Call `generate_story()` to create narration and dialogue.
- Call `generate_image()` for each panel.
- Call `build_comic_layout()` to organize the content.
- Call `save_pdf()` to create the downloadable PDF.

8. Routes and Frontend Integration

Backend Route Development

The route handlers connect user actions to the development workflow. The main routes provide different entry points for browser-based and API-based interaction.

Route	Development Function
<code>/</code>	Loads the homepage and input form.
<code>/generate</code>	Receives form input and performs complete comic generation.
<code>/generate-comic/json</code>	Accepts JSON input and returns layout data and PDF path.
<code>/test-image</code>	Tests image generation directly.
<code>/export-success</code>	Displays export confirmation.

Frontend Development

The frontend is developed using HTML, CSS, and Jinja2. The interface provides a simple flow from entering comic details to viewing the generated comic and confirming the PDF export.

Frontend Templates

- `index.html` – Collects story prompt, main character name, setting, story tone, and art style.
- `comic_preview.html` – Displays the generated comic panel-by-panel.
- `export_success.html` – Shows a confirmation after the comic is downloaded.

Dynamic Rendering

FastAPI's Jinja2Templates is used to dynamically render the HTML templates. The `/` route renders `index.html`, the `/generate` route renders `comic_preview.html`, and the `/export-success` route renders `export_success.html`. Generated images, panel titles, scene descriptions, captions, and narration are passed from the backend to the templates.

9. Complete Development Flow and Outcome

End-to-End Development Flow

1. The user enters the required comic details through the homepage.
2. FastAPI receives the form or JSON input.
3. Gemini Flash generates a structured five-panel outline.
4. Gemini Pro generates detailed narration and character dialogues.
5. Stable Diffusion creates an illustration for each panel.
6. The layout builder matches images with their corresponding story content.
7. FPDF compiles the comic into a multi-page PDF.
8. The generated comic is displayed through the Jinja2 frontend.
9. The user can download the completed comic and receive an export-success confirmation.

Development Integration

The development phase integrates the AI models, backend services, frontend templates, image generation, layout processing, and PDF export into one application workflow. The project document describes the backend as the coordinator between user input, AI generation, layout construction, and frontend rendering.

Expected Result

At the end of development, ComicCraft provides an end-to-end comic-generation experience. A user can provide a creative idea and preferences, receive a panel-by-panel comic with narration and illustrations, preview the result, and export it as a PDF.

Conclusion

The Project Development Phase converts the planned ComicCraft architecture into working functionality. The implementation of Gemini-based storytelling, Stable Diffusion image generation, FastAPI routing, Jinja2 templates, layout building, and PDF export forms the core of the application.

Source: ComicCraft – AI Comic Story Creator using Gemini Models project document.